

Retrieval Augmented Generation (RAG)

Jorge Ortiz Fuentes

10 de junio de 2025

Introducción

Orígenes y evolución del RAG

Arquitectura del RAG

Ventajas de RAG

Desventajas y desafíos de RAG

Componentes clave y técnicas avanzadas

Panorama actual: discusiones y debates

Introducción

¿Qué es RAG?

Definición

- RAG (*Retrieval-Augmented Generation*) combina LLMs preentrenados con recuperación de información externa
- Los LLMs estándar se basan en conocimiento estático de su entrenamiento
- RAG recupera fragmentos relevantes y los usa para enriquecer la respuesta (Lewis et al., 2020)

Importancia

Mejora la factualidad, la relevancia y permite citar fuentes

Mejoras y problemas persistentes en LLMs

Avances recientes

- *Instruction tuning*
- *Reinforcement learning from human feedback (RLHF)*
- *Ingeniería de Prompting*

Desafíos actuales

- *Hallucination* (invención de información)
- *Attribution* (rastreo del origen de la información)
- *Staleness* (conocimiento desactualizado)
- *Revisions y Customization* (actualización y adaptación)

RAG surge como una solución para estos desafíos

Formas de conceptualizar el conocimiento

- ***Closed book vs. Open book*** (Roberts et al., 2020):
 - *Closed book*: LM estándar memoriza conocimiento en sus parámetros
 - *Open book*: RAG accede a una base de datos externa para obtener conocimiento
- ***Parametric vs. Semi-parametric*** (Yogatama et al., 2021):
 - *Parametric*: Conocimiento reside enteramente en los parámetros del modelo
 - *Semi-parametric*: RAG combina LLM (paramétrico) con un retriever (no paramétrico)

Orígenes y evolución del RAG

Hito principal

- RAG fue formalizado y popularizado por Lewis et al. (2020)
- Demostraron superioridad en tareas intensivas en conocimiento
- Tiene raíces en sistemas de QA anteriores

El desafío de ODQA

- Responder preguntas sobre una amplia gama de temas, basándose en un gran corpus de conocimiento (ej. Wikipedia)
- Modelos pre-RAG enfrentaban limitaciones:
 - Conocimiento estático (limitado a sus datos de entrenamiento)
 - Tendencia a alucinar (inventar respuestas)
 - Dificultad para actualizar conocimiento o incorporar información nueva
 - Poca interpretabilidad sobre el origen de la respuesta

Arquitectura y contribuciones

- Propusieron explícitamente la arquitectura RAG para tareas intensivas en conocimiento
- Combinaron un *retriever* no paramétrico con un generador paramétrico (*seq2seq*, BART)
- El *retriever* busca información relevante en Wikipedia y el generador condiciona su respuesta

Impacto

- Mejora significativa en la factualidad de las respuestas
- Reducción de alucinaciones al basar la generación en evidencia explícita

Arquitectura del RAG

Flujo habitual de RAG

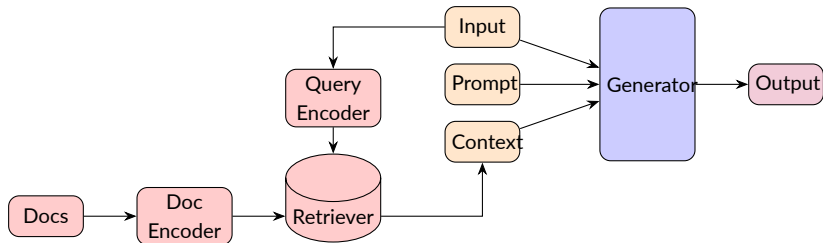


Figura 1: Arquitectura RAG (adaptada de clase de Stanford CS25 V3 sobre *Retrieval Augmented Language Models*)

Recuperador (Retriever)

- Codificación de documentos y consultas en vectores (embeddings).
- Construcción de un índice *Approximate Nearest Neighbor* (ANN) para permitir búsquedas eficientes.
- Gestión de un datastore con pares clave-valor que asocia cada embedding con su fragmento de texto o documento completo.
- Proceso de consulta: codificar la pregunta, realizar búsqueda ANN para obtener los k fragmentos más relevantes, y aplicar post-procesamiento (re-ranking, filtrado) si es necesario.

Generador (Generator)

- LLM preentrenado, generalmente basado en la arquitectura Transformer.
- Se encarga de generar la respuesta final basándose en la consulta y la información recuperada.
- Los generadores se pueden clasificar en dos categorías:
 - **Generadores por defecto:** La mayoría de los LLMs preentrenados, como los de las series GPT, Mistral y Gemini.
 - **Generadores aumentados por recuperación (RA):** Modelos preentrenados que incluyen módulos específicos para fusionar la información recuperada, como RETRO (Borgeaud et al., 2021).

Ventajas de RAG

Beneficios clave de RAG (I)

Conocimiento actualizado y específico

- Acceso a información no presente en datos de entrenamiento (ej. datos recientes, de dominio específico)
- No se requiere reentrenar (*customization*, combate *staleness*)

Reducción de alucinaciones

- Fundamentar respuestas en evidencia recuperada disminuye significativamente la invención de información y reduce alucinaciones (Dai et al., 2024; Lewis et al., 2020; Shuster et al., 2021)

Beneficios clave de RAG (II)

Transparencia y explicabilidad

- Permite citar fuentes, facilitando la verificación y comprensión del origen de la información (*attribution*)
- Aumenta la confianza e interpretabilidad

Actualización y control

- Actualizar la base de conocimiento externa es más económico y rápido que reentrenar un LLM (*revisions*)
- Mayor control sobre la información utilizada, facilitando personalización

Desventajas y desafíos de RAG

Dependencia de la calidad de recuperación

- La efectividad de RAG está ligada a la calidad del *retriever*
- Información irrelevante o incorrecta en la recuperación afecta la respuesta final (Dai et al., 2024)

Latencia y complejidad

- El proceso de recuperación introduce latencia adicional
- Implementar y mantener pipelines RAG robustos puede ser complejo

Sesgos y costos

- Posibilidad de heredar y amplificar sesgos de la base de conocimiento externa
- Costos computacionales significativos (recuperación e inferencia con prompts grandes)

Manejo de información

- Dificultad para manejar fragmentos recuperados que son irrelevantes, contradictorios o ruidosos

Componentes clave y técnicas avanzadas

Áreas de optimización e investigación

La efectividad de RAG depende de la sofisticación de sus componentes y las técnicas empleadas.

- El recuperador (*retriever*)
- El generador (*generator*)
- La ingeniería de *prompts*

Retriever: tipos de *embeddings* y técnicas (I)

Sparse Retrieval

- Basados en conteo de palabras
- Bueno para coincidencias exactas
- Ejemplos: TF-IDF, BM25

Dense Retrieval

- Capturan similitud semántica
- Puede fallar en palabras clave exactas
- Ejemplos: Sentence Transformers, OpenAI Embeddings

Hybrid Search

Suele ser la mejor opción práctica, combinando fortalezas de *sparse* y *dense retrieval*

Bases de datos vectoriales: almacenamiento y búsqueda

- Una base de datos vectorial está diseñada para almacenar, gestionar y buscar datos codificados como vectores.
- Se componen de tres elementos clave:
 - **Chunking y Encoding:** Los documentos se dividen en fragmentos (*chunks*) y se codifican como vectores.
 - **Indexing:** Se construye un índice especializado para acelerar la búsqueda, utilizando algoritmos de ANN (por ejemplo, IVFPQ, HNSW, Annoy).
 - **Datastore (base de datos vectorial):** Almacena los datos originales (ej. el texto de los *chunks*) asociados a sus vectores, a menudo como pares clave-valor.

Bases de datos vectoriales open source

- **FAISS:** Biblioteca de Meta en C++/Python para búsquedas de similitud en vectores a gran escala, con soporte GPU.
- **Annoy:** Biblioteca basada en árboles de partición aleatorios (random projection trees) en C++/Python, ligeros y mmap.
- **Qdrant:** Base de datos con motor en Rust que ofrece búsqueda de similitud, cuantización y persistencia de vectores para entornos escalables.
- **pgvector:** Extensión de PostgreSQL que añade un tipo vectorial y funciones ANN para RAG.

Retriever: Re-ranking para mejorar la precisión

¿Qué es el re-ranking?

Es una segunda etapa de filtrado que refina los resultados del Retrieval para asegurar que se seleccionen los documentos más relevantes.

Proceso en dos fases

1. **Recuperación inicial (*Retriever*):** Un modelo rápido recupera un conjunto amplio de documentos potencialmente relevantes (ej. los 20-100 mejores). Se prioriza la **exhaustividad (*recall*)**.
2. **Re-clasificación (*Re-ranker*):** Un modelo más potente y lento evalúa la relevancia de cada documento recuperado con respecto a la consulta y los reordena. Se prioriza la **precisión** (Humeau et al., 2020).

Ejemplo de re-ranking

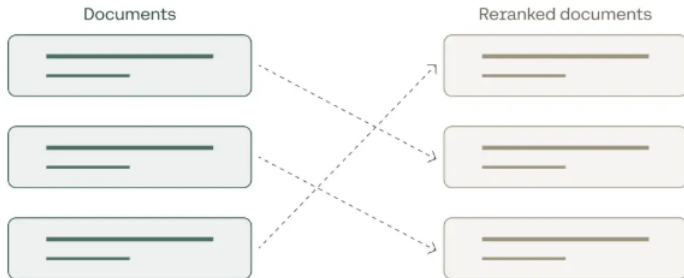


Figura 2: Ejemplo de re-ranking en RAG (extraído de post de Cohere sobre Rerank)

Rol del generador

- Típicamente un LLM potente (GPT, Claude, Gemini, DeepSeek, etc.)
- Sintetiza una respuesta coherente y precisa basada en la consulta original y la información recuperada
- Capacidad de integrar y razonar sobre la información recuperada es crucial

Ejemplo de prompt para RAG

Prompt ejemplo

You are an assistant for question-answering tasks. Use the following retrieved context to answer the question. If you don't know the answer, just say that you don't know. Keep it under three sentences.

Question: {question}

Context: {context}

Answer:

Fuente: LangChain Prompt Hub

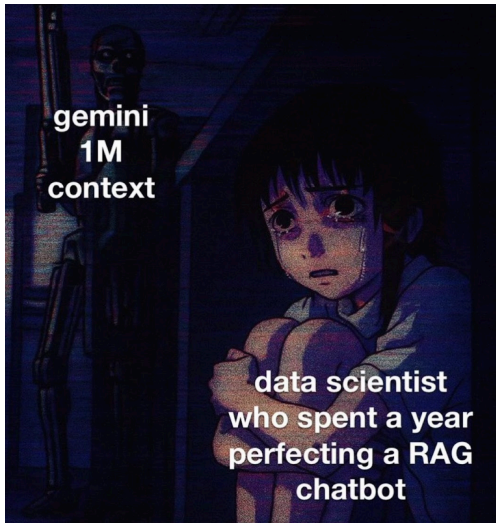
Impacto del *prompt*

- La estructura del *prompt* (consulta + chunks recuperados) afecta significativamente la calidad de la respuesta
- Un buen *prompt* guía al LLM a enfocarse, citar fuentes, manejar contradicciones (Zhang et al., 2024)

Panorama actual: discusiones y debates

El campo de RAG está en evolución y desarrollo.

RAG vs. Ventanas de Contexto Largas (LCWs)



RAG vs. Long Context Windows (LCWs)

LCWs

- Conceptualmente simples si el conocimiento cabe en el *prompt*
- Pueden sufrir de *lost in the middle* (Liu et al., 2023)
- Más costoso
- No resuelven conocimiento dinámico o muy masivo

RAG

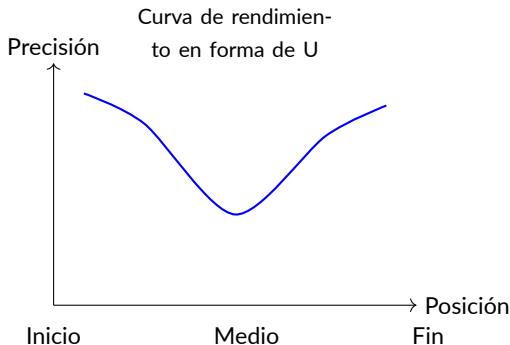
- Ventajoso para bases de conocimiento masivas y dinámicas
- Mayor precisión en algunos casos
- Control granular y eficiencia de costos

RAG y LCWs no son mutuamente excluyentes ya que se pueden combinar.

El problema de “Lost in the Middle”

Investigación de Liu et al. (2023)

- LLMs tienden a prestar más atención a la información al principio y al final del contexto largo
- La información en el medio a menudo se ignora o se pierde



Niveles de implementación (basado en Dai et al. (2024))

- ***Naive RAG:***
 - Implementación básica: indexación, recuperación, generación
 - *Chunking* simple, *embeddings* genéricos
- ***Advanced RAG:***
 - Optimizaciones en pre/post-procesamiento y recuperación (ej. re-ranking)
- ***RAG modular y agéntico:***
 - Arquitecturas flexibles, módulos intercambiables (ej. Self-RAG (Asai et al., 2023))

Evaluación de sistemas RAG

Midiendo el rendimiento de RAG

El RAG puede evaluarse tanto desde el *retrieval* como desde la *generación*.

Definición

- Mide la proporción de documentos relevantes que son recuperados entre los K primeros resultados
- Es importante cuando se quiere asegurar que la mayoría de los documentos relevantes sean encontrados

Fórmula

$$\text{Recall@K} = \frac{\text{Nº de docs recuperados en el top K}}{\text{Número total de documentos relevantes}}$$

Definición

- Mide la proporción de documentos recuperados en los K primeros resultados que son realmente relevantes
- Es crucial cuando la exactitud de los primeros resultados es prioritaria

Fórmula

$$\text{Precision@K} = \frac{\text{Nº de docs recuperados en el top K}}{K}$$

Definición

- Indica la proporción de consultas para las cuales al menos un documento relevante se encuentra entre los K primeros resultados recuperados
- Es una métrica útil para entender si el sistema suele encontrar algo relevante rápidamente

Fórmula

$$\text{Hit Rate@K} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \mathbb{I}(\text{hit}_i@K)$$

Donde $|Q|$ es el número total de consultas y $\mathbb{I}(\text{hit}_i@K)$ es 1 si la consulta i tiene al menos un resultado relevante en el top K, y 0 en caso contrario.

Mean Reciprocal Rank (MRR)

Definición

- Evalúa la calidad de un sistema de recuperación basándose en la posición del primer resultado relevante
- Es el promedio de la inversa del rango del primer resultado correcto para un conjunto de consultas

Fórmula

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Donde $|Q|$ es el número total de consultas y rank_i es la posición del primer documento relevante para la consulta i . Si no hay documentos relevantes para una consulta, $\frac{1}{\text{rank}_i}$ se considera 0 para esa consulta.

Dimensiones de calidad

- **Fidelidad/Veracidad (*Faithfulness/Factuality*)**: adhesión al contexto recuperado
- **Relevancia de la respuesta (*Answer Relevance*)**: pertinencia a la consulta del usuario
- **Coherencia y fluidez**: calidad lingüística de la respuesta
- **No toxicidad / sesgos**: ausencia de lenguaje ofensivo o sesgos

RAG Multimodal (MM-RAG)

Extendiendo RAG más allá del texto

Concepto de MM-RAG

- El conocimiento humano es multimodal (texto, imágenes, audio, video)
- MM-RAG busca extender el paradigma RAG para recuperar y razonar sobre información de múltiples modalidades

Desafíos significativos

- Creación de *embeddings* multimodales efectivos
- Diseño de arquitecturas de recuperación para datos modales múltiples
- Capacidad de LLMs para fusionar y generar desde información multimodal

Arquitectura de RAG Multimodal

El flujo de MM-RAG adapta el pipeline tradicional para manejar datos heterogéneos.

1. **Consulta multimodal:** el usuario puede preguntar usando texto, una imagen, o una combinación de ambas.
2. **Recuperación multimodal:**
 - Se usan **embeddings multimodales** para codificar la consulta y los documentos (imágenes, texto, audio) en un espacio semántico compartido.
 - Se buscan los fragmentos más relevantes en una base de datos vectorial multimodal.
3. **Generación multimodal:** un **LLM Multimodal (MLLM)** procesa la consulta y el contexto recuperado para generar una respuesta.

Componente clave: Embeddings Multimodales

Definición

Son representaciones vectoriales que capturan la semántica de diferentes modalidades en un espacio compartido (Radford et al., 2021).

¿Cómo se crean?

- Generalmente con aprendizaje contrastivo a gran escala.
- Se entrena un modelo para maximizar la similitud entre pares correctos (ej. imagen y su leyenda) y minimizarla para pares incorrectos.

Modelos notables

- **CLIP** (OpenAI): Pionero en alinear texto e imágenes.
- **ImageBind** (Meta): Unifica 6 modalidades (texto, imagen/video, audio, etc.) en un solo espacio.

Componente clave: LLMs Multimodales (MLLM)

¿Qué son los MLLMs?

Son LLMs extendidos para procesar, comprender y generar contenido a partir de múltiples modalidades.

Arquitectura y modelos líderes

- **Arquitectura:** usualmente, un codificador de modalidad (ej. un Vision Transformer) proyecta la información no textual a un espacio que el LLM puede entender.
- **Empresas y modelos:**
 - **OpenAI:** GPT-4V, GPT-4o y modelos en adelante
 - **Google:** Gemini 1.5 en adelante
 - **Meta:** Llama 3.2 (con capacidades multimodales)

Frameworks y herramientas

Herramientas populares

- **LangChain:** framework más popular para construir aplicaciones LLM, incluyendo pipelines RAG complejos
- **LlamaIndex:** framework especializado la preparación e indexación de datos
- **Haystack:** framework end-to-end listo para producción de RAG/búsqueda semántica con énfasis en escalabilidad y monitorización.

Direcciones futuras e investigación abierta

Algunas líneas de investigación actuales (I)

- **RAG Agéntico:** Combinar RAG con agentes LLM autónomos para problemas complejos (ej: Deep Research)
- **GraphRAG:** Usar grafos de conocimiento como fuente externa para razonar sobre relaciones
- **Auto-mejora y aprendizaje por retroalimentación:** Sistemas RAG que aprenden de errores (ej. Asai et al. (2023))

Algunas líneas de investigación actuales (II)

- Optimización de costos, latencia y escalabilidad para bases de conocimiento enormes
- Uso de RAG para diferentes task de NLP
- Mejoras en robustez y fiabilidad: manejo de información ruidosa, contradictoria o desactualizada

Referencias



Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H.
(2023). **Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.** *arXiv preprint*
arXiv:2310.11511. <https://arxiv.org/abs/2310.11511>



Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J., Damoc, B., Clark, A., de Las Casas, D., Guy, A., Menick, J., Ring, R., Hennigan, T., Huang, S., Maggiore, L., Jones, C., Cassirer, A., ... Sifre, L. (2021). **Improving language models by retrieving from trillions of tokens.** *CoRR*, abs/2112.04426. <https://arxiv.org/abs/2112.04426>



Dai, Z., Zheng, Z. V., Zhang, A., Zhao, Z., Zhang, J., Zhou, H., Yin, D., & Singh, A. (2024). **Retrieval-Augmented Generation for Natural Language Processing: A Survey.** *arXiv preprint arXiv:2407.13193*. <https://arxiv.org/abs/2407.13193>



Humeau, S., Shuster, K., Lachaux, M.-A., & Weston, J. (2020).
**Poly-encoders: Transformer Architectures and
Pre-training Strategies for Fast and Accurate
Multi-sentence Scoring.** <https://arxiv.org/abs/1905.01969>



Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N.,
Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T.,
Riedel, S., & Kiela, D. (2020). **Retrieval-Augmented
Generation for Knowledge-Intensive NLP Tasks.**
Advances in Neural Information Processing Systems, 33,
9459-9474. [https://proceedings.neurips.cc/paper/2020/
hash/6b493230205f780e1bc26945df7481e5-Abstract.html](https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html)



Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). **Lost in the Middle: How Language Models Use Long Contexts.**

<https://arxiv.org/abs/2307.03172>



Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). **Learning Transferable Visual Models From Natural Language Supervision.**

<https://arxiv.org/abs/2103.00020>

-  Roberts, A., Raffel, C., & Shazeer, N. (2020). **How Much Knowledge Can You Pack Into the Parameters of a Language Model?** *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 5418-5426.
-  Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). **Retrieval Augmentation Reduces Hallucination in Conversation.** *arXiv preprint arXiv:2104.07567*.
<https://arxiv.org/abs/2104.07567>
-  Yogatama, D., d'Áutume, C. d. M., Connor, J., Daulton, S., Gu, K., Graves, A., & Legassick, R. (2021). **Adaptive Computation Tradeoffs for Conditional Language Models.** *arXiv preprint arXiv:2110.09380*. <https://arxiv.org/abs/2110.09380>



Zhang, C., Zhao, W. X., Wang, S., & Wen, J.-R. (2024). **Towards Understanding Retrieval Accuracy and Prompt Quality in RAG Systems.** *arXiv preprint arXiv:2411.19463*.
<https://arxiv.org/abs/2411.19463>